SoftwareX 34 (2026) 102556

Contents lists available at ScienceDirect

SoftwareX

journal homepage: www.elsevier.com/locate/softx

Original software publication

MRAC-LLM Toolbox: An interactive model reference adaptive control enhanced with large language models

Merve Nilay Aydın [a,*], Halil Ibrahim Okur [b], Handan Gürsoy-Demir [b], Kadir Tohma [b], Celaleddin Yeroğlu [b]

[a] Iskenderun Technical University, Management Information Systems, Iskenderun, Türkiye
[b] Iskenderun Technical University, Computer Engineering, Iskenderun, Türkiye

ARTICLE INFO

Keywords:
Model reference adaptive control
Large language models
Adaptive control
Control system design
Parameter optimization
Human-in-the-loop control
Interactive software
Open source

ABSTRACT

This paper presents the MRAC-LLM Toolbox, a MATLAB-based software that integrates classical Model Reference Adaptive Control (MRAC) with Large Language Model (LLM) assistance for interactive control system design. The software enables users to specify performance requirements through a graphical user interface, which are translated into reference model configurations with LLM guidance. MRAC-LLM focuses exclusively on classical MRAC and supports simulation and real-time parameter adaptation within a master–slave framework. The LLM operates strictly at an advisory level, assisting with reference model selection and adaptation guidance without modifying the underlying MRAC control laws or their stability guarantees. The software features a modular architecture supporting simulation, visualization, and reporting, and is released as open-source under the MIT License at https://github.com/halilokur91/MRAC_LLM.

Code metadata

Nr.	Code metadata description	Metadata
C1	Current code version	v1.0
C2	Permanent link to code/repository used for this code version	https://github.com/halilokur91/MRAC_LLM
C3	Permanent link to Reproducible Capsule	https://doi.org/10.5281/zenodo.17649905
C4	Legal Code License	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	MATLAB, Simulink, OpenAI API
C7	Compilation requirements, operating environments & dependencies	MATLAB R2019b or later, Simulink, Control System Toolbox, OpenAI API key
C8	If available Link to developer documentation/manual	https://github.com/halilokur91/MRAC_LLM/blob/main/README.md
C9	Support email for questions	hibrahim.okur@iste.edu.tr

1. Motivation and significance

Model Reference Adaptive Control is a well-established adaptive control framework that enables dynamical systems to track desired reference behavior in the presence of parametric uncertainties and unmodeled dynamics. Strong stability and convergence guarantees, typically derived using Lyapunov-based analysis, have led to successful applications of MRAC in aerospace systems, robotics, and industrial automation [1–5,7,8].

Despite this strong mathematical complexity, the practical adoption of MRAC remains limited. Reference model selection and adaptation gain tuning often require advanced expertise and extensive trial-and-error [9,10,26]. In practice, control objectives are commonly expressed using intuitive performance metrics such as overshoot, settling time, or transient smoothness; however, translating these specifications into mathematically consistent state-space reference models is not directly supported by conventional MRAC design tools [11,12].

* Corresponding author.
Email address: mnilay.aydin@iste.edu.tr (M.N. Aydın).

https://doi.org/10.1016/j.softx.2026.102556
Received 25 January 2026; Accepted 7 February 2026
Available online 12 February 2026
2352-7110/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Recent advances in LLMs offer new opportunities to address this challenge by interpreting high-level performance requirements and mapping them to structured design representations [19,21,22,25]. However, the direct use of LLMs as autonomous control elements raises concerns related to stability, safety, and interpretability [15,16]. Human-in-the-loop and advisory paradigms have gained increasing attention [13,14].

Motivated by these observations, MRAC-LLM is designed as an interactive and reproducible software framework that lowers the barrier to classical MRAC design without compromising theoretical rigor. The system employs LLMs strictly as an advisory layer to support reference model selection and parameter tuning, while preserving classical MRAC control laws and stability guarantees [1,5]. Through a user-friendly MATLAB App Designer interface and a modular MATLAB/Simulink implementation, MRAC-LLM enables intuitive performance-based specification, visualization, and reproducible experimentation, aligning with the goals of SoftwareX for accessible and reusable scientific software [24]. Building on our prior work in SmartControl [23], this study extends LLM-assisted interactive design from fixed-structure PID controllers to adaptive control systems.

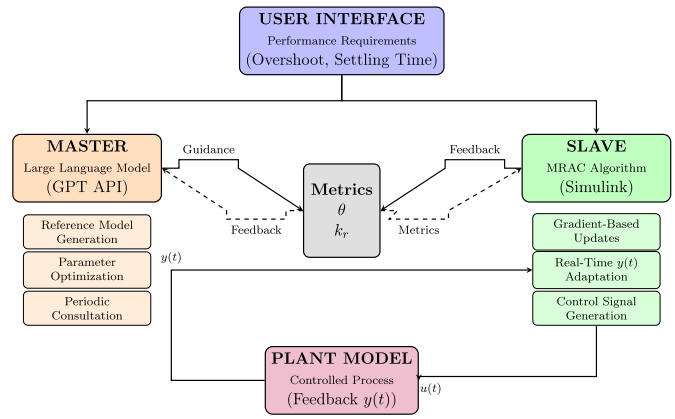

Fig. 1. MRAC-LLM system architecture implementing the Master–Slave framework.

2. Related work

The theoretical foundations of MRAC are well established through seminal works that rigorously analyze stability, convergence, and robustness using Lyapunov-based methods [1–5]. Extensions in nonlinear and adaptive control have further generalized these principles to broader system classes [6,12]. Comprehensive surveys have reviewed MRAC developments and applications, particularly in robotic and aerospace systems, while consistently emphasizing challenges related to reference model selection and adaptation tuning [7–9].

While numerous studies improve MRAC robustness and transient performance through techniques such as composite and proportional-integral adaptation laws [10,11], practical implementation remains limited due to reference model selection and tuning complexity. Control objectives are typically expressed using intuitive performance metrics such as overshoot, settling time, or transient smoothness, whereas MRAC design requires mathematically consistent state-space reference models and carefully tuned adaptation gains. Conventional MRAC frameworks offer limited support for translating such high-level intent into formal design parameters, creating a significant barrier for non-expert users and interdisciplinary applications [7–9].

Human-centered control and human-in-the-loop design paradigms have therefore gained increasing attention as a means of addressing this gap. These approaches emphasize shared autonomy, decision support, transparency, and user trust without replacing classical control laws [13,14]. Parallel developments in explainable AI further highlight the importance of interpretability and transparency in safety-critical control systems [17,18], aligning naturally with the use of intelligent advisory tools to augment classical control structures.

More recently, LLMs have emerged as promising instruments for interpreting high-level technical requirements and generating structured design recommendations [19–22]. Initial studies have explored LLM-guided adaptive control, agentic control frameworks, and LLM-based controller optimization [27–30]. While the direct deployment of LLMs as control policies remains limited due to safety and stability concerns, their use as design-time or supervisory assistants complements human-in-the-loop paradigms.

Within the SoftwareX literature, interactive and reproducible control design platforms have been proposed to improve accessibility to advanced control techniques. The SmartControl framework [23] demonstrated that LLM-assisted interaction can support PID controller design while preserving classical control principles. However, existing tools predominantly address fixed-structure controllers and do not tackle the unique challenges of adaptive control systems such as MRAC. In contrast, MRAC-LLM extends this interactive, LLM-assisted design philosophy to classical MRAC, employing LLMs strictly as advisory components for reference model selection and parameter tuning, while preserving theoretical guarantees through an interpretable and reproducible software framework.

3. Software description

MRAC-LLM is a MATLAB-based software system that integrates classical MRAC with LLM assistance to support intuitive adaptive control system design. The software enables users to specify performance requirements, such as overshoot and settling time, through a graphical interface, which are translated into candidate reference model configurations with LLM guidance. MRAC-LLM focuses exclusively on classical MRAC and is released as open-source software under the MIT License at https://github.com/halilokur91/MRAC_LLM, together with comprehensive documentation and example models.

The software is designed with a strong emphasis on usability, reproducibility, and theoretical rigor. LLM assistance is limited to an advisory role, preserving classical MRAC stability guarantees. Efficient Simulink execution, real-time visualization, and structured error handling contribute to a streamlined user workflow, guiding users from initial configuration to simulation and result analysis.

3.1. Software architecture

MRAC-LLM is built on a modular software architecture that clearly separates user interaction, control algorithms, and LLM-based advisory components. As illustrated in Fig. 1, the system adopts a Master–Slave strategy in which the LLM provides high-level guidance, while the classical MRAC algorithm performs real-time adaptation and control signal generation.

As shown in Fig. 1, the architecture implements the Master–Slave framework, where the Master (LLM) generates reference model configurations based on user performance requirements and provides periodic parameter optimization guidance. The Slave (MRAC) performs real-time gradient-based adaptation and control signal generation. Metrics (θ, k_r) are exchanged between components to enable collaborative refinement while maintaining classical MRAC stability guarantees. The architecture consists of five main functional layers communicating through well-defined interfaces, as summarized in Table 1.

The User Interface layer, implemented using MATLAB App Designer, provides a tabbed layout that guides users through reference model configuration, simulation execution, performance analysis, and result reporting, as detailed in Table 2.

This architectural design ensures that LLM assistance is limited to an advisory role, preserving classical MRAC stability guarantees. The modular MATLAB/Simulink integration provides a flexible and reproducible

Table 1
MRAC-LLM architecture layers and their responsibilities.

Layer	Implementation	Key responsibilities
User Interface	MRACApp.m	Tabbed GUI for model configuration, simulation, analysis, reporting, and settings
MRAC Core	mrac_combined.m, mrac_classic.slx	Classical MRAC algorithms, Lyapunov-based adaptation laws, real-time parameter updates
LLM Interface	callGptApi_combined.m	LLM-based advisory support for reference model selection and adaptation guidance
Simulation Engine	runMRAC-Simple.m	Coordination of Simulink execution and data synchronization
Output Generation	reportUtils.m	Visualization, performance analysis, and report generation

Table 2
User interface tabs and their functionalities.

Tab name	Functionality
Home	Documentation access and quick start guidance
Model Selection	Reference and plant model configuration with LLM assistance
Simulation	Real-time visualization of system response and adaptation behavior
Analysis	Conversational performance analysis and parameter guidance via LLM
Reporting	Export of simulation results in HTML and PDF formats
Settings	Configuration of API keys and LLM models

software framework that supports extensibility and efficient adaptive control system design.

3.2. Software functionalities

MRAC-LLM Toolbox provides a set of integrated functionalities that support the interactive design, simulation, and analysis of classical MRAC systems. The main functionalities are summarized as follows:

Performance-based model selection: Users specify control objectives through an intuitive graphical interface by selecting desired performance characteristics such as overshoot and settling time. The system queries the LLM to generate multiple candidate reference model configurations in state-space form, which users can review and select or manually define custom models.

Classical MRAC implementation: The software implements standard MRAC algorithms grounded in well-established adaptive control theory [1,3,5], using MATLAB/Simulink models that incorporate plant dynamics, reference models, tracking error computation, and Lyapunov-based adaptation laws.

Real-time simulation with LLM-guided adaptation: MRAC-LLM supports real-time simulation using a master–slave strategy where the classical MRAC algorithm performs fast local parameter updates while the LLM periodically provides high-level advisory guidance. System responses, tracking errors, and parameter evolution are visualized in real time.

Interactive analysis and parameter guidance: An integrated conversational interface allows users to query control performance, request design adjustments, and obtain explanatory feedback using natural language.

Automated reporting and visualization: The software generates structured reports containing simulation results, time-domain responses, parameter evolution, and quantitative performance metrics, exportable in HTML and PDF formats.

3.3. MRAC structure

MRAC-LLM implements the classical Model Reference Adaptive Control framework [1,3,5,9]. Following the standard formulation [9], the plant dynamics are:

$$\dot{x}(t) = Ax(t) + Bu(t) + B\Delta(x(t)) \tag{1}$$

where $x(t) \in \mathbb{R}^n$ is the state vector, $u(t) \in \mathbb{R}^m$ is the control signal, and $\Delta(x(t)) = -W^T\beta(x(t))$ represents matched uncertainty with an unknown weight matrix $W \in \mathbb{R}^{s\times m}$ and a known regressor $\beta : \mathbb{R}^n \to \mathbb{R}^s$.

The reference model is:

$$\dot{x}_m(t) = A_m x_m(t) + B_m r(t) \tag{2}$$

where A_m is Hurwitz. The control signal consists of nominal and adaptive components:

$$u(t) = -K_1 x(t) + K_2 r(t) + \hat{W}^T(t)\beta(x(t)) \tag{3}$$

where K_1 and K_2 are chosen such that $A_m = A - BK_1$ and $B_m = BK_2$, and $\hat{W}(t)$ is the adaptive parameter matrix.

The tracking error $e(t) = x_m(t) - x(t)$ is minimized using the adaptation law:

$$\dot{\hat{W}}(t) = \Gamma_0\beta(x(t))e^T(t)PB \tag{4}$$

where $\Gamma_0 > 0$ is the adaptation gain matrix, and $P > 0$ satisfies the Lyapunov equation:

$$A_m^T P + PA_m = -Q \tag{5}$$

for $Q > 0$. This ensures asymptotic stability [1,9]. Fig. 2 presents the proposed LLM-supervised MRAC framework with LLM-guided reference model selection and adaptation parameter tuning.

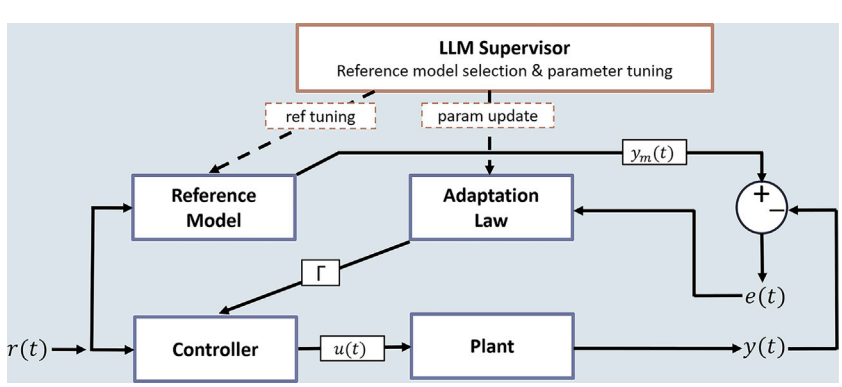

Fig. 2. Interactive chat interface for qualitative performance analysis and advisory feedback.

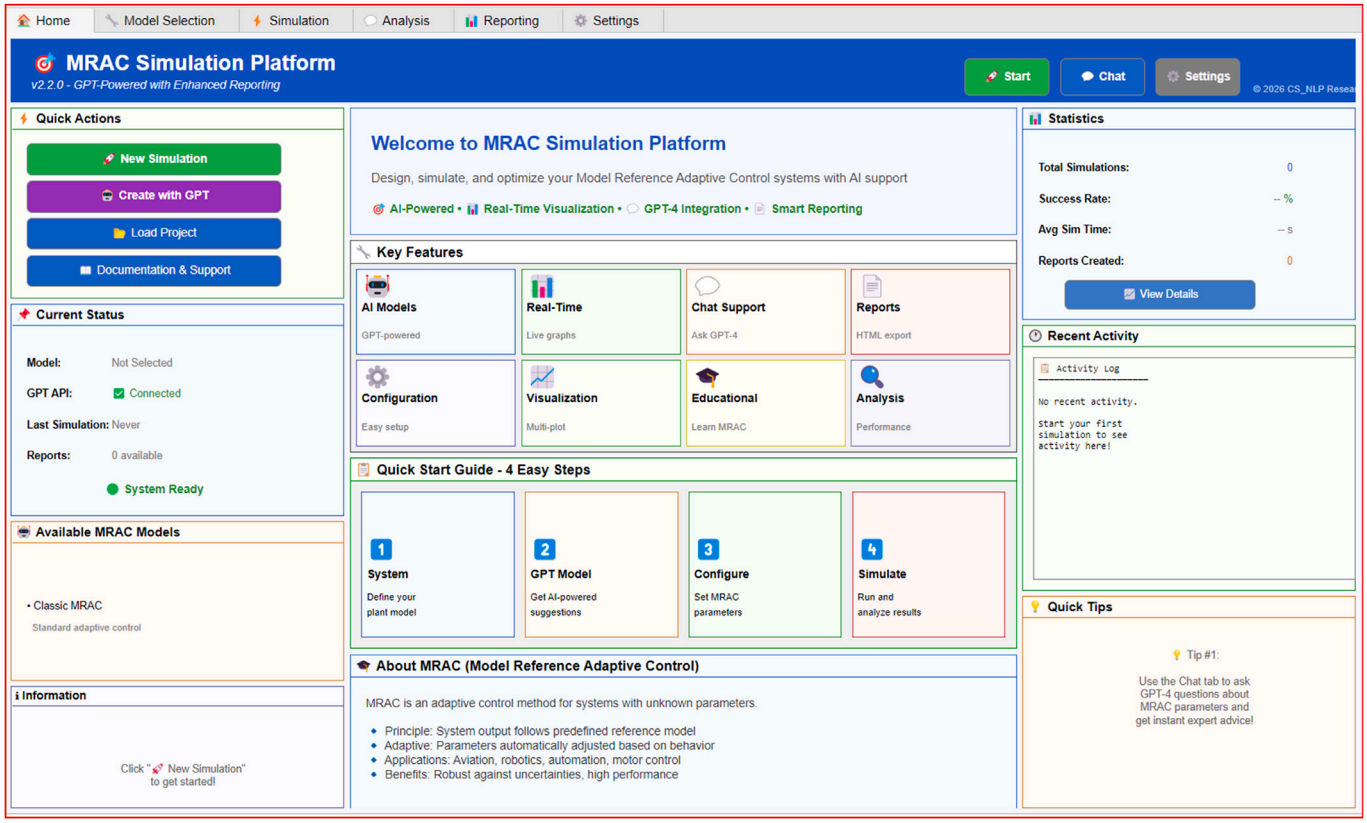

Fig. 3. Main graphical user interface of MRAC-LLM showing the overall workflow and navigation structure.

3.4. Software implementation overview

MRAC-LLM is implemented using a modular and maintainable software structure that clearly separates user interaction, control algorithms, external services, and output generation. This design enables independent development and modification of individual components without affecting the overall system functionality.

Main application structure: The core application is implemented using MATLAB App Designer (`MRACApp.m`), which manages the graphical user interface, user interactions, and system state. The application follows a structured design with dedicated sections for initialization, callback functions, and utility methods, enabling intuitive user interaction and simplified code maintenance.

LLM integration: LLM functionality is encapsulated in dedicated MATLAB functions responsible for reference model selection and adaptation guidance. These modules handle communication with the OpenAI API, process user-defined performance requirements, and return structured advisory outputs. Robust error handling ensures stable operation and graceful fallback to classical MRAC adaptation when LLM services are unavailable.

Simulink-based MRAC core: The adaptive control algorithms are implemented using Simulink models, which represent plant dynamics, reference models, tracking error computation, and Lyapunov-based adaptation laws. A MATLAB bridge function coordinates data exchange between the GUI and the Simulink environment, allowing control researchers to modify MRAC structures directly at the model level without interacting with the user interface code.

Reporting and documentation: Reporting functionality is implemented as a standalone module that generates structured HTML and PDF reports containing simulation results, visualizations, and performance metrics. In addition, the software includes comprehensive documentation of LLM configuration parameters and usage guidelines within the repository.

Pre-configured example models: The repository provides a collection of pre-configured example models covering a range of control scenarios. These examples allow users to quickly initiate simulations, explore different system behaviors, and use existing configurations as templates for custom adaptive control designs.

4. Illustrative examples

This section presents illustrative examples demonstrating the core functionalities of the MRAC-LLM Toolbox and its interactive workflow. The examples highlight how LLM-assisted model selection, classical MRAC execution, and adaptive parameter tuning are combined within a unified software environment.

4.1. End-to-end workflow

Fig. 3 illustrates the main graphical user interface, which provides access to all stages of the MRAC design process, including model definition, LLM-assisted reference model selection, simulation, performance analysis, and report generation.

4.2. LLM-assisted reference model selection

As shown in Fig. 4, users specify high-level performance requirements via dropdown-based selections. The LLM generates multiple candidate reference model configurations in state-space form, representing different trade-offs between transient speed and smoothness.

4.3. Adaptive simulation and parameter refinement

Fig. 5 presents representative simulation results obtained using the selected reference model. The system executes classical MRAC in real time, while the LLM provides periodic high-level guidance through a master–slave strategy. Between consultations, local gradient-based

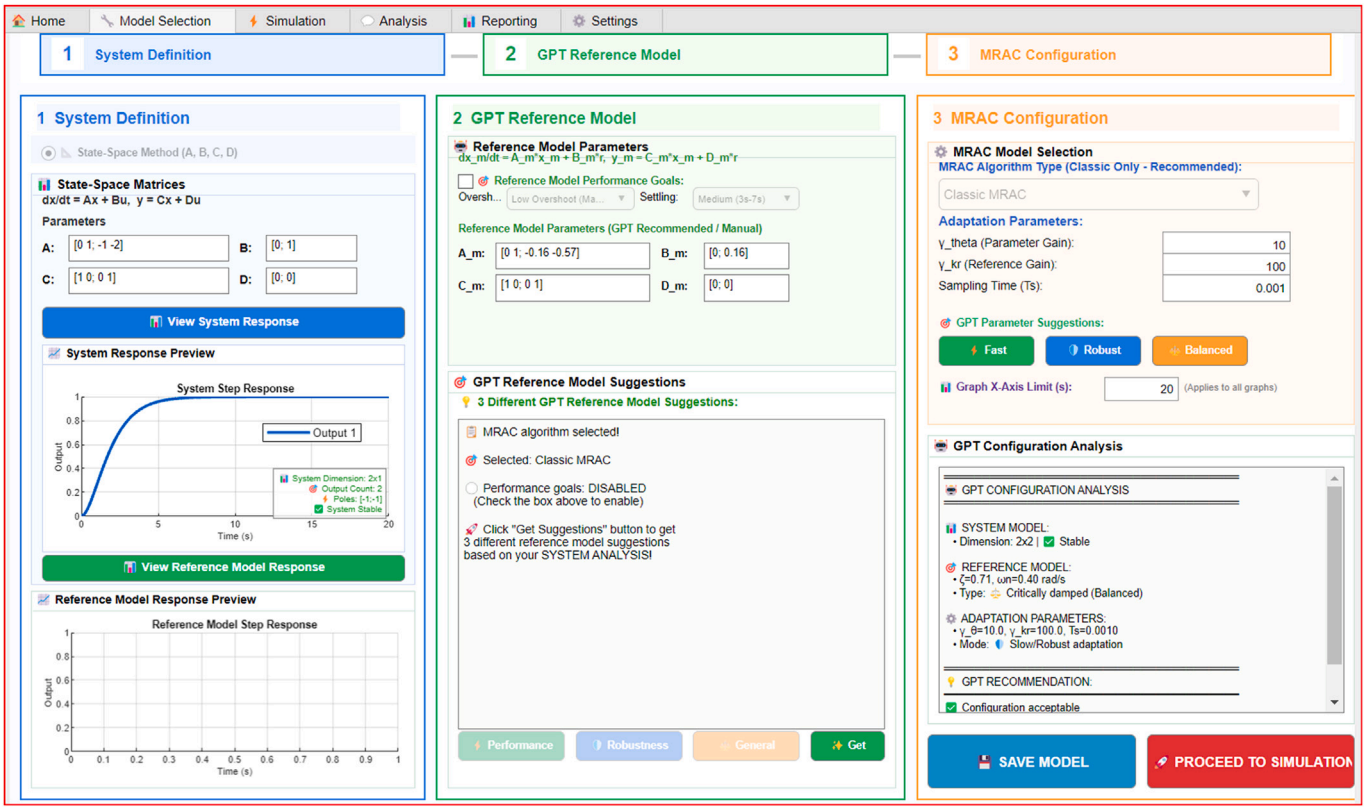

Fig. 4. Model Selection tab with LLM-assisted reference model proposals.

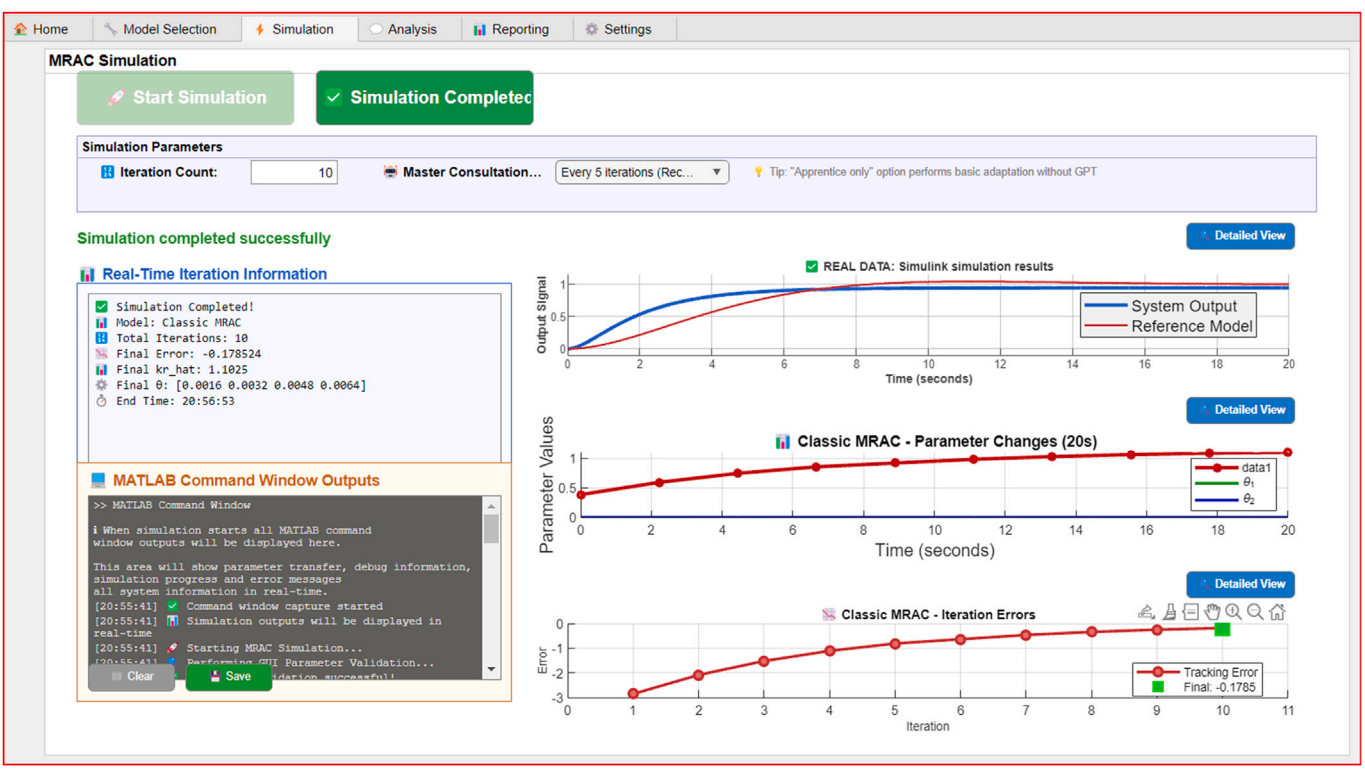

Fig. 5. Simulation results illustrating system response and adaptive parameter evolution.

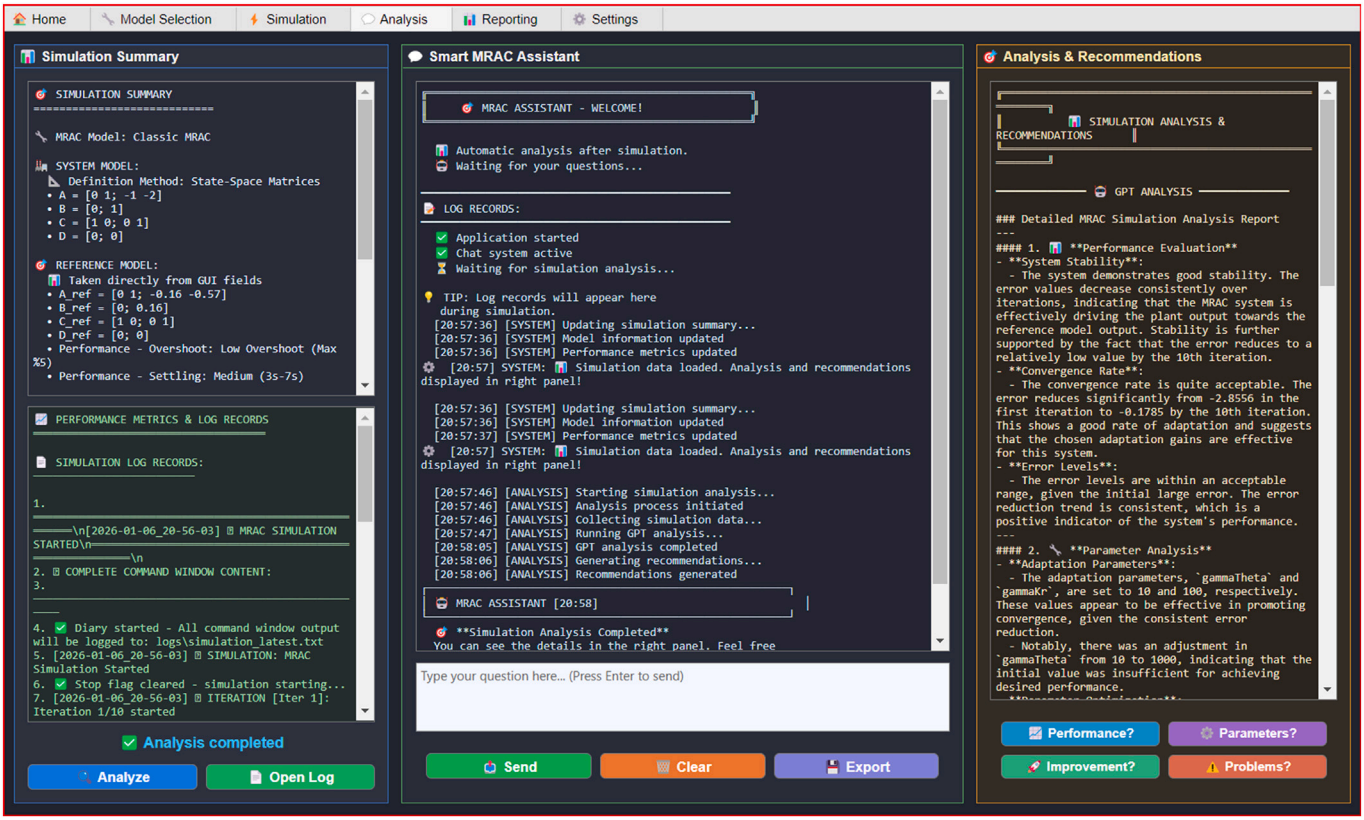

Fig. 6. Interactive chat interface for qualitative performance analysis and advisory feedback.

adaptation ensures fast and stable parameter updates. This hybrid approach improves transient behavior while preserving the theoretical guarantees of classical MRAC.

In addition, the integrated chat interface shown in Fig. 6 enables users to query system behavior and request qualitative adjustments using natural language. The LLM responds with advisory feedback, supporting informed decision-making without directly altering the underlying control laws.

5. Impact

MRAC-LLM Toolbox advances the practice of classical adaptive control by introducing an interactive, LLM-assisted software framework that lowers the barrier to MRAC design. LLM assistance is limited to an advisory role, preserving classical MRAC stability guarantees.

Research impact: MRAC-LLM opens new research directions at the intersection of adaptive control, human–machine interaction, and AI-assisted system design. It provides a reproducible platform for investigating performance-based reference model generation, master–slave adaptation strategies, and the role of explainability in adaptive control workflows.

Practical and educational impact: The intuitive, dropdown-based interface reduces the expertise required to design MRAC systems, making classical adaptive control accessible to non-specialists, students, and interdisciplinary researchers. By translating high-level performance specifications into reference model configurations, MRAC-LLM allows users to focus on system analysis rather than manual parameter derivation.

Adoption and reuse: The open-source release under the MIT License, together with comprehensive documentation and MATLAB/Simulink compatibility, supports adoption in both academic and industrial contexts. MRAC-LLM can be integrated into existing control design workflows and extended to new MRAC variants without modification of core components. Potential application domains include aerospace, robotics, industrial automation, and automotive systems.

6. Conclusions

This paper introduces the MRAC-LLM Toolbox, an interactive software framework that integrates LLM with classical MRAC while preserving theoretical rigor and stability guarantees. By translating intuitive performance specifications into reference model configurations through LLM-assisted guidance, the software addresses one of the main practical challenges of MRAC design: reference model selection and parameter tuning.

The proposed system adopts a human-centered design philosophy where LLM assistance is limited to an advisory role, preserving classical MRAC stability guarantees. The modular MATLAB/Simulink implementation, together with real-time simulation and visualization capabilities, provides a reproducible and extensible platform for adaptive control system design and analysis.

Limitations and future work: The current implementation focuses on classical MRAC formulations. Future work will extend the framework to filtered, time-delay, and robust MRAC variants, explore adaptive strategies for optimizing master–slave consultation frequency, and investigate local or hybrid deployment solutions for LLM services.

Overall, MRAC-LLM contributes a practical and transparent software foundation that bridges high-level performance intent and classical adaptive control design. The open-source release at https://github.com/halilokur91/MRAC_LLM encourages community-driven development and future extensions in AI-assisted adaptive control.

CRediT authorship contribution statement

Merve Nilay Aydın: Methodology, Software, Writing – original draft. **Halil Ibrahim Okur:** Data curation, Formal analysis,

Methodology, Software, Validation, Visualization, Writing – original draft. **Handan Gürsoy-Demir:** Investigation. **Kadir Tohma:** Investigation, Visualization. **Celaleddin Yeroğlu:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors would like to thank the Iskenderun Technical University for providing the computational resources and support necessary for this research. Special thanks to the OpenAI team for providing access to their API services, which enabled the LLM integration capabilities of the software.

References

[1] Narendra KS, Annaswamy AM. Stable adaptive systems. Prentice Hall; 1989.
[2] Åström KJ, Wittenmark B. Adaptive control. 2nd ed. Dover Publications; 2013.
[3] Ioannou PA, Sun J. Robust adaptive control. Dover Publications; 2012.
[4] Lavretsky E, Wise KA. Robust and adaptive control: with aerospace applications. Springer; 2013. https://doi.org/10.1007/978-1-4471-4396-3
[5] Sastry S, Bodson M. Adaptive control: stability, convergence, and robustness. Prentice Hall; 1989.
[6] Slotine J-JE, Li W. Applied nonlinear control. Prentice Hall; 1991.
[7] Zhang D, Wei B. A review on model reference adaptive control of robotic manipulators. Annu Rev Control 2017;43:188–98. https://doi.org/10.1016/j.arcontrol.2017.02.002
[8] Shekhar A, Sharma A. Review of model reference adaptive control. In: Proc. ICICET. IEEE; 2018. https://doi.org/10.1109/ICICET.2018.8533774
[9] Kökçam E, Tan N. Comparison of adaptation mechanisms on MRAC. IEEE Access 2024;12:1–15. https://doi.org/10.1109/ACCESS.2024.3367215
[10] Lavretsky E. Combined/composite model reference adaptive control. IEEE Trans Autom Control 2011;56(11):2692–7. https://doi.org/10.1109/TAC.2011.2161096
[11] Ioannou PA, Kokotović PV. Adaptive systems with reduced models. Springer; 1996.
[12] Krstić M, Kanellakopoulos I, Kokotović P. Nonlinear and adaptive control design. Wiley; 1995.
[13] Endsley MR. From Automation to autonomy. Human Factors 2017;59(1):5–27. https://doi.org/10.1177/0018720816681350
[14] Sheridan TB. Human–robot interaction: status and challenges. Hum Factors 2016;58(4):525–32. https://doi.org/10.1177/0018720816644364
[15] García J, Fernández F. A comprehensive survey on safe reinforcement learning. J Mach Learn Res 2015;16:1437–80.
[16] Amodei D, Olah C, Steinhardt J, Christiano P, Schulman J, Mané D. Concrete problems in AI safety. [arXiv preprint] arXiv:1606.06565. 2016.
[17] Hoffman RR, et al. Metrics for explainable AI. [arXiv preprint] arXiv:1812.04608. 2018.
[18] Doshi-Velez F, Kim B. Towards a rigorous science of interpretable machine learning. [arXiv preprint] arXiv:1702.08608. 2017.
[19] Brown TB, et al. Language models are few-shot learners. NeurIPS 2020;33:1877–901.
[20] Bommasani R, et al. On the opportunities and risks of foundation models. [arXiv preprint] arXiv:2108.07258. 2021.
[21] Liu P, Yuan W, Fu J, Jiang Z, Hayashi H, Neubig G. Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing. ACM Comput Surv 2023;55(9):1–35.
[22] Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, Zhang C, Agarwal S, Slama A, Ray A, et al.). Training language models to follow instructions with human feedback. In: Advances in neural information processing systems, vol. 35: Curran Associates, Inc; 2022. pp. 27730–44.
[23] Tohma K, Okur HİI, Gürsoy-Demir H, Aydın MN, Yeroğlu C. SmartControl: interactive PID controller design powered by LLM agents. SoftwareX 2025;31:102194. https://doi.org/10.1016/j.softx.2025.102194
[24] Wilkinson MD, Dumontier M, Aalbersberg IJ, Appleton G, Axton M, Baak A, et al. The FAIR guiding principles for scientific data management and stewardship. Sci Data 2016;3:160018. https://doi.org/10.1038/sdata.2016.18
[25] Anisuzzaman DM, Malins JG, Friedman PA, Attia ZI. Fine-tuning large language models for specialized use cases. Mayo Clin Proc: Digit Health 2025;3(1):100184.
[26] Gramuglia M, Kumar GM, L'Afflitto A. Potential and challenges for a certified application of model reference adaptive control to aerial vehicles. In: 2024 IEEE international workshop on technologies for defense and security (TechDefense). IEEE; Nov 2024. p. 425–9.
[27] Zhou Z, Lu Y, Zhu Y, Zhao Y, He B, He L. et al. LLMs-guided adaptive compensator: Bringing Adaptivity to Automatic Control Systems with Large Language Models. arXiv preprint arXiv:2507.20509. 2025.
[28] Arif TM, Rahim MA. Agentic AI for Real-Time adaptive PID control of servo motor. In: Actuators, vol. 14: MDPI. Sep 2025. p. 459, No. 9.
[29] Narimani M, Emami SA. AgenticControl: An Automated Control Design Framework Using Large Language Models. [arXiv preprint] arXiv:2506.19160. 2025.
[30] Kamenko I, Ilic S, Congradac V. LLM-based PID controller optimization. In: 2025 10th international conference on smart and sustainable technologies (SpliTech). IEEE; Jun 2025. p. 1–6.